SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4301
Name:Reddykonda Thrisha
Reg no:192373045

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

(a) Analyze the possible reasons for lack of responsiveness in the existing layout.
(b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
(c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
(d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
(e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

(a) Analyze the issues in the existing form validation mechanism.
(b) Design input fields for email and phone number with appropriate HTML attributes. (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
(d) Develop a mechanism to display **real-time error messages** for invalid inputs.
(e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

(a) Analyze the CSS properties responsible for layout overlapping issues.
(b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
(c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
(e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

I T Gnaneswari(192372183)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

T Gnaneswari

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1: RESPONSIVE LAYOUT DESIGN

AIM

To design a responsive retail webpage using HTML5, CSS Flexbox, CSS Grid, and media queries so that the layout adapts correctly to mobile, tablet, and desktop screen sizes.

PROBLEM STATEMENT

A retail website is not responsive on mobile and tablet devices. The navigation links overflow, product cards become misaligned, images extend outside their containers, and improper spacing affects usability.

The webpage must be redesigned so that the header, navigation bar, content, product cards, and footer remain properly aligned on different screen sizes.

(a) ANALYSIS OF THE POSSIBLE REASONS FOR LACK OF RESPONSIVENESS

The main reasons for a non-responsive webpage are:

1. Missing viewport meta tag.
2. Use of fixed widths such as *width: 1200px*.
3. Excessive use of absolute positioning.
4. Absence of media queries.
5. Fixed image dimensions.
6. Flexbox items without wrapping.
7. Large margins and padding.
8. Navigation links overflowing on mobile devices.
9. Large fixed font sizes.
10. Missing *box-sizing: border-box*.

The viewport meta tag is required to match the webpage width with the device width.

<meta name="viewport"

content="width=device-width, initial-scale=1.0">

Fixed width: .container { width: 1200px;

}

Responsive width:

.container { width:

90%; max-width:

1200px;

}

Images should also be made responsive.

img { width:

100%;

height: auto;

}

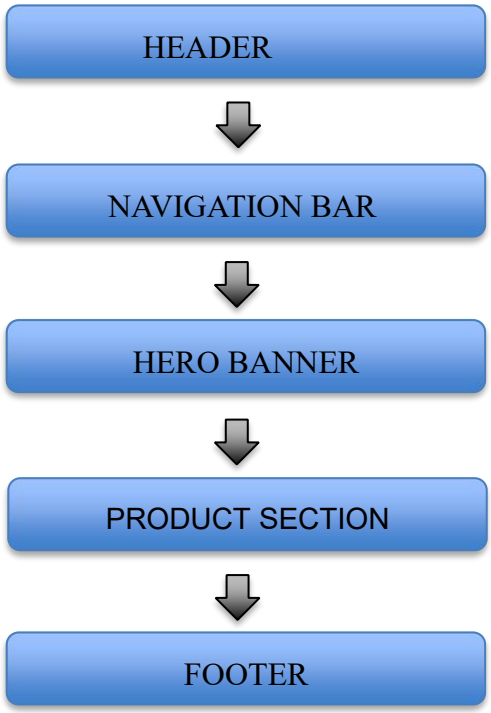
(b) RESPONSIVE WEBPAGE DESIGN

The webpage contains:

11. Header with logo and cart button.
12. Navigation bar.
13. Hero banner.
14. Product section.
15. Product cards.
16. Service features.
17. Footer.

Flexbox is used for the header, navigation bar, and services. CSS Grid is used to arrange the product cards.

Layout Structure



ALGORITHM

18. Start the program.
19. Create the HTML document structure.
20. Add the viewport meta tag.
21. Create the header and navigation bar.
22. Use Flexbox for alignment.
23. Create the hero section.
24. Add product cards using CSS Grid.
25. Apply proper margin, padding, and gap.
26. Add media queries for tablet and mobile screens.
27. Test the webpage by resizing the browser.
28. Stop the program.

PROGRAM

Save the file as *responsive_retail.html*.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">

   <meta name="viewport"
        content="width=device-width, initial-scale=1.0">

   <title>Responsive Retail Website</title>

   <style>
      * {          margin: 0;
padding: 0;          box-sizing:
border-box;
      }

      body {
```

```
        font-family: Arial, sans-serif;
background-color: #f4f6f8;          color:
#222;
        line-height: 1.6;
    }

    .container {
width: 90%;          max-
width: 1100px;
margin: auto;
    }

    header {          padding: 18px
0;          background-color:
#17324d;          color: white;
    }

    .header-content {          display:
flex;          justify-content: space-
between;          align-items: center;
gap: 20px;
    }

    .logo {          font-
size: 28px;          font-
weight: bold;
    }

    .cart-button {
padding: 10px 18px;
border: none;          border-
radius: 5px;          background-
color: #ff8c42;          color:
white;          font-size: 16px;
cursor: pointer;
```

```
    }

    nav {           background-
color: #29465f;
    }

    .nav-links {
display: flex;          justify-
content: center;          flex-
wrap: wrap;          list-style:
none;
    }

    .nav-links a {
display: block;
padding: 14px 22px;
color: white;          text-
decoration: none;
    }

    .nav-links a:hover {
background-color: #ff8c42;
    }

    .hero {          min-height:
320px;          display: flex;
justify-content: center;
align-items: center;
padding: 40px 20px;
background-color: #dce8f2;
text-align: center;
    }

    .hero h1 {
```

```
            margin-bottom: 12px; color:
            #17324d;
            font-size: clamp(30px, 5vw, 52px);
        }

        .hero p {            margin-
bottom: 20px;            font-
size: 18px;
        }

        .shop-button {
display: inline-block;
padding: 11px 24px;
border-radius: 5px;
background-color: #ff8c42;
color: white;            text-
decoration: none;
        }

        main {
padding: 40px 0;
        }

        .section-title {            margin-
bottom: 28px;            text-align:
center;
        }

        .product-grid {
display: grid;            grid-
template-columns:
                repeat(auto-fit, minmax(220px, 1fr));
gap: 22px;
        }
```

```
        .product-card {
overflow: hidden;
background-color: white;
border-radius: 8px;             box-
shadow: 0 4px 12px
                    rgba(0, 0, 0, 0.15);
        }

        .product-image {
height: 180px;             display:
flex;             justify-content: center;
align-items: center;
background-color: #dbe7f1;
color: #17324d;              font-size:
22px;             font-weight: bold;
        }

        .product-details {
padding: 18px;
        }

        .product-details h3 {
margin-bottom: 8px;              color:
#17324d;
        }

        .product-details p {
margin-bottom: 10px;              color:
#555;
        }

        .price { display:
            block; margin-
            bottom: 12px;
            color:
```

```
        #d35400;
        font-size:
        20px;
        font-weight:
        bold;
    }

    .buy-button {        width:
100%;        padding: 10px;
border: none;        border-radius:
5px;        background-color:
#17324d;        color: white;
cursor: pointer;
    }

    .features {        margin-top:
40px;        padding: 25px;
background-color: #e4ebf1;
border-radius: 8px;
    }

    .feature-container {
display: flex;        justify-content:
space-between;        flex-wrap:
wrap;        gap: 18px;
    }

    .feature-box {        flex:
1 1 200px;        padding:
20px;        background-color:
white; border-radius: 6px;
        text-align: center;
    }
```

```
        footer {            padding: 20px;
background-color: #17324d;
color: white;            text-align:
center;
        }

        /* Tablet layout */
        @media screen and (max-width: 992px) {
            .product-grid {
                grid-template-columns: repeat(2, 1fr);
            }
        }

        /* Mobile layout */
        @media screen and (max-width: 600px) {
            .header-content {
flex-direction: column;
            }

            .nav-links {
flex-direction: column;
text-align: center;
            }

            .product-grid {                grid-
template-columns: 1fr;
            }

            .feature-container {
flex-direction: column;
            }

            .hero {
min-height: 260px;
            }
```

```
        }
    </style>
</head>

<body>

<header>
    <div class="container header-content">
        <div class="logo">Smart Retail</div>

        <button class="cart-button">
            Shopping Cart
        </button>
    </div>
</header>

<nav>
    <ul class="nav-links">
        <li><a href="#home">Home</a></li>
        <li><a href="#products">Products</a></li>
        <li><a href="#offers">Offers</a></li>
        <li><a href="#features">Services</a></li>
        <li><a href="#contact">Contact</a></li>
    </ul>
</nav>

<section class="hero" id="home">
    <div>
        <h1>Shop Smarter, Live Better</h1>

        <p>
            Discover quality products at affordable prices.
        </p>

        <a href="#products" class="shop-button">
```

```
            Explore Products
        </a>
    </div>
</section>

<main>
    <div class="container">

        <div class="section-title" id="products">
            <h2>Featured Products</h2>

            <p>Browse our latest products and offers.</p>
        </div>

        <div class="product-grid">

            <div class="product-card">
                <div class="product-image">Smart Watch</div>

                <div class="product-details">
                    <h3>Smart Watch</h3>

                    <p>
                        Track health, steps and daily activities.
                    </p>

                    <span class="price">₹2,499</span>

                    <button class="buy-button">
                        Add to Cart
                    </button>
                </div>
            </div>

            <div class="product-card">
```

```
    <div class="product-image">
      Headphones
    </div>

    <div class="product-details">
      <h3>Wireless Headphones</h3>

      <p>
        Clear sound with long battery life.
      </p>

      <span class="price">₹1,899</span>

      <button class="buy-button">
        Add to Cart
      </button>
    </div>
  </div>

  <div class="product-card">
    <div class="product-image">
      Sports Shoes
    </div>

    <div class="product-details">
      <h3>Sports Shoes</h3>

      <p>
        Comfortable and lightweight footwear.
      </p>

      <span class="price">₹1,599</span>

      <button class="buy-button">
        Add to Cart
```

```
            </button>
        </div>
    </div>

    <div class="product-card">
        <div class="product-image">
            Fitness Band
        </div>

        <div class="product-details">
            <h3>Fitness Band</h3>

            <p>
                Monitor sleep, calories and heart rate.
            </p>

            <span class="price">₹1,199</span>

            <button class="buy-button">
                Add to Cart
            </button>
        </div>
    </div>

</div>

<section class="features" id="features">
    <div class="feature-container">

        <div class="feature-box">
            <h3>Free Delivery</h3>
            <p>Free delivery on selected orders.</p>
        </div>

        <div class="feature-box">
```

```
                <h3>Secure Payment</h3>
                <p>Safe and protected payment process.</p>
            </div>

            <div class="feature-box">
                <h3>Easy Returns</h3>
                <p>Simple replacement and return process.</p>
            </div>

        </div>
    </section>

  </div>
</main>

<footer id="contact">
  <p>© 2026 Smart Retail. All rights reserved.</p>
</footer>

</body>
</html>
```

EXPLANATION OF THE PROGRAM

The universal selector applies *box-sizing: border-box* to all elements. This prevents padding and borders from increasing the specified width.

```
* {    box-sizing: border-
box;
}
```

The header uses Flexbox to place the logo and cart button on opposite sides.

```
.header-content {
display: flex;
  justify-content: space-between;
  align-items: center;
}
```

The navigation bar also uses Flexbox. The *flex-wrap* property allows links to move to the next line when screen space is limited.

```
.nav-links {     display:
flex;     justify-content:
center;     flex-wrap:
wrap;
}
```

The product section uses CSS Grid.

```
.product-grid {     display:
grid;     grid-template-
columns:
        repeat(auto-fit, minmax(220px, 1fr));
gap: 22px;
}
```

The *auto-fit* property adjusts the number of columns automatically. The *minmax()* function gives each card a minimum width of 220 pixels.

(c) ADAPTATION USING MEDIA QUERIES

Desktop View

On desktop screens:

- The logo and cart button appear in one row.
- Navigation links are horizontal.
- Multiple product cards appear in each row.
- Feature boxes are displayed side by side.

Tablet View

The tablet media query displays two product cards in each row.

```
@media screen and (max-width: 992px) {
   .product-grid {
      grid-template-columns: repeat(2, 1fr);
   }
}
```

Mobile View

The mobile media query changes the layout into a single-column format.

```
@media screen and (max-width: 600px) {
   .header-content {         flex-
direction: column;
```

```
    }

    .nav-links {        flex-
direction: column;
    }

    .product-grid {        grid-template-
columns: 1fr;
    }
}
```

On mobile devices:

- Header elements appear vertically.
- Navigation links are displayed one below another.
- One product card appears in each row.
- Feature boxes are stacked vertically.
-

(d) SPACING, ALIGNMENT AND DISTRIBUTION

The following CSS properties improve visual consistency:

29. *gap* provides equal spacing between items.
30. *padding* creates space inside cards and sections.
31. *margin* creates space outside elements.
32. *justify-content* distributes Flexbox items.
33. *align-items* controls vertical alignment.
34. *1fr* creates equal-width Grid columns.
35. *max-width* prevents content from becoming too wide.
36. *margin: auto* centres the main container.

Example:

```
.product-grid {
gap: 22px;
}

.product-details {
padding: 18px;
}
```

```
.container {
margin: auto;
}
```

TEST CASES

Test Case	Expected Result
Desktop view	Multiple product cards appear in one row
Tablet view	Two product cards appear in each row
Mobile view	One product card appears in each row
Navigation test	Links appear vertically on mobile
Header test	Logo and cart button stack on mobile
Resize test	No content overlaps
Overflow test	No horizontal scrolling occurs
Feature test	Feature boxes stack on small screens

EXPECTED OUTPUT

Desktop Output

The desktop view displays a horizontal header, navigation bar, hero section, multiple product cards, service features and footer.

Tablet Output

The tablet view displays two product cards per row with proper spacing.

Mobile Output

The mobile view displays vertical navigation, one product card per row and vertically arranged service boxes.

(e) EVALUATION AND IMPROVEMENTS

The responsive webpage adjusts correctly to desktop, tablet and mobile screen sizes. Flexbox maintains the alignment of the header and navigation bar, while CSS Grid automatically arranges the product cards.

The design prevents content overlap and horizontal scrolling. Consistent use of margin, padding and gap improves the appearance of the webpage.

Further improvements include:

37. Add a hamburger navigation menu.
38. Add product search and filtering.
39. Add working shopping-cart functionality.
40. Use responsive product images.

41. Add lazy loading for faster performance.
42. Add accessibility labels.
43. Add keyboard navigation.
44. Test the design in different browsers.

ADVANTAGES

45. The webpage works on different screen sizes.
46. Flexbox simplifies alignment.
47. Grid organizes product cards properly.
48. Media queries provide device-specific layouts.
49. Responsive dimensions prevent overlapping.
50. The design is easy to maintain.
51. Proper spacing improves readability.

LIMITATIONS

52. The cart button does not contain complete cart functionality.
53. Product search and filtering are not included.
54. The webpage contains only front-end implementation.
55. The mobile navigation takes more vertical space.

RESULT

Thus, a responsive retail webpage was successfully designed using HTML5, CSS Flexbox, CSS Grid and media queries.

The webpage adapts properly to mobile, tablet and desktop screen sizes without overlapping, misalignment or horizontal scrolling.

EXPERIMENT 2: FORM VALIDATION USING JAVASCRIPT

AIM

To design a registration form and validate the email address and phone number using JavaScript regular expressions, with real-time error messages.

PROBLEM STATEMENT

A registration form accepts invalid email addresses and phone numbers, resulting in incorrect information being stored. JavaScript validation is used to prevent invalid data submission.

(a) ANALYSIS OF EXISTING FORM VALIDATION ISSUES

The major issues in an improperly validated registration form are:

56. The form accepts empty input fields.
57. The email field accepts text without @ and a valid domain.
58. The phone field accepts letters and special characters.
59. The phone number may contain fewer or more than ten digits.
60. Error messages are displayed only after form submission.
61. Client-side validation may be completely absent.
62. Invalid data may be stored in the database.

HTML validation and JavaScript regular expressions can be used to solve these problems.

(b) HTML INPUT FIELDS

The email input uses the *type="email"* attribute, and the phone-number input uses *type="tel"*, *maxlength*, and *pattern* attributes.

(c) JAVASCRIPT VALIDATION USING REGULAR EXPRESSIONS

The email is checked using an email regular expression. The phone number is validated as a ten-digit Indian mobile number beginning with 6, 7, 8, or 9.

ALGORITHM

1. Start the program.
2. Create a registration form.
3. Read the name, email, and phone number.
4. Check whether all fields are filled.
5. Validate the email using a regular expression.
6. Validate the phone number using a regular expression.
7. Display an error message when an input is invalid.
8. Submit the form only when all fields are valid.
9. Stop the program.

PROGRAM

Save the file as *form_validation.html*.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
<meta name="viewport"
        content="width=device-width, initial-scale=1.0">

   <title>Registration Form Validation</title>

   <style>
      * {
         box-sizing: border-box;
      }

      body {          margin: 0;
min-height: 100vh;          display:
flex;          justify-content: center;
align-items: center;          font-
family: Arial, sans-serif;
background-color: #e9f1f7;
      }

      .form-container {          width: 90%;
max-width: 420px;          padding: 30px;
background-color: white;          border-radius:
10px;          box-shadow: 0 5px 18px rgba(0, 0,
0, 0.2);
      }
       h2 {          text-
align: center;
color: #173b57;
      }
```

```
.form-group { margin-bottom: 18px;
}
label { display: block; margin-bottom: 6px; font-weight: bold;
}
input { width: 100%; padding: 11px; border: 1px solid #999; border-radius: 5px; font-size: 16px;
}

input:focus { border-color: #1769aa; outline: none;
}

.error { display: block; min-height: 18px; margin-top: 4px; color: red; font-size: 13px;
}

.valid { color: green;
}

button { width: 100%; padding: 12px; border:
```

```
none;            border-radius: 5px;
background-color: #1769aa;
color: white;            font-size: 16px;
cursor: pointer;
      }

      button:hover {
background-color: #124f80;
      }

      #successMessage {
margin-top: 15px;            text-
align: center;            color:
green;            font-weight:
bold;
      }
   </style>
</head>

<body>

<div class="form-container">
   <h2>Registration Form</h2>

   <form id="registrationForm" novalidate>

      <div class="form-group">
         <label for="name">Name</label>

         <input type="text"
id="name"                  placeholder="Enter
your name"                  required>

         <span class="error" id="nameError"></span>
      </div>
```

```
      <div class="form-group">
        <label for="email">Email Address</label>

        <input type="email"
id="email"
            placeholder="example@gmail.com"
required>

        <span class="error" id="emailError"></span>
      </div>

      <div class="form-group">
        <label for="phone">Phone Number</label>          <input type="tel"
id="phone"                maxlength="10"
            placeholder="Enter 10-digit number"
pattern="[6-9][0-9]{9}"                required>

        <span class="error" id="phoneError"></span>
      </div>

      <button type="submit">Register</button>

      <p id="successMessage"></p>
    </form>
</div>

<script>
    const form = document.getElementById("registrationForm");
const nameInput = document.getElementById("name");    const
emailInput = document.getElementById("email");    const
phoneInput = document.getElementById("phone");

    const nameError = document.getElementById("nameError");
const emailError = document.getElementById("emailError");
```

```
const phoneError = document.getElementById("phoneError");
const successMessage =
      document.getElementById("successMessage");

   const emailPattern =
      /^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;

   const phonePattern = /^[6-9][0-9]{9}$/;

   function validateName() {        const
name = nameInput.value.trim();        if
(name.length < 3) {
nameError.textContent =
            "Name must contain at least 3 characters.";
return false;
      }

      nameError.textContent = "Valid name";
nameError.className = "error valid";        return
true;
   }

   function validateEmail() {        const
email = emailInput.value.trim();

      if (!emailPattern.test(email)) {
emailError.textContent =
"Enter a valid email address.";
emailError.className = "error";
return false;
      }

      emailError.textContent = "Valid email address";
emailError.className = "error valid";        return
true;
```

```
    }

    function validatePhone() {        const
phone = phoneInput.value.trim();

        if (!phonePattern.test(phone)) {
phoneError.textContent =
                "Enter a valid 10-digit mobile number.";
phoneError.className = "error";            return
false;
        }

        phoneError.textContent = "Valid phone number";
phoneError.className = "error valid";        return
true;
    }

    nameInput.addEventListener("input", validateName);
emailInput.addEventListener("input", validateEmail);
phoneInput.addEventListener("input", validatePhone);

    form.addEventListener("submit", function (event) {
event.preventDefault();

        const nameValid = validateName();
const emailValid = validateEmail();        const
phoneValid = validatePhone();

        if (nameValid && emailValid && phoneValid) {
successMessage.textContent =                "Registration
completed successfully!";            form.reset();

            nameError.textContent = "";
emailError.textContent = "";
phoneError.textContent = "";
```

```
    } else {
        successMessage.textContent = "";
    }
  });
</script>

</body>
</html>
```

(d) REAL-TIME ERROR MESSAGE MECHANISM

The *input* event is used to validate the values whenever the user types.

emailInput.addEventListener("input", validateEmail); phoneInput.addEventListener("input", validatePhone);

When the input is invalid, a red error message is displayed. When the input is valid, a green confirmation message is displayed.

REGULAR EXPRESSIONS USED

Email validation

/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/

This expression checks the username, @ symbol, domain name, and domain extension.

Phone-number validation

/^[6-9][0-9]{9}$/

This expression accepts exactly ten digits and requires the first digit to be between 6 and 9.

TEST CASES

Input	Expected Result
gnaneswari@gmail.com	Valid email
gnaneswari@gmail	Invalid email
9876543210	Valid phone number
1234567890	Invalid phone number

98765abc10	Invalid phone number
Empty input	Error message displayed

(e) EVALUATION AND ENHANCEMENTS

The validation system prevents incorrect email and phone-number inputs. Real-time messages help users correct errors before submitting the form.

Further enhancements include:

1. Validate the information again on the server.
2. Add OTP verification for email and phone numbers.
3. Prevent repeated form submissions.
4. Use CAPTCHA to prevent automated registrations.
5. Sanitize all user inputs before database storage.
6. Use HTTPS for secure data transmission.

RESULT

Thus, a registration form was created and successfully validated using HTML attributes, JavaScript regular expressions, and real-time error messages.

EXPERIMENT 3: CSS LAYOUT DEBUGGING

AIM

To analyze and debug a webpage that overlaps when the browser window is resized and to correct the layout using the CSS box model, Flexbox, Grid, and responsive properties.

PROBLEM STATEMENT

A webpage contains sections positioned using fixed sizes and absolute positioning. When the browser window is resized, the content overlaps and affects usability.

(a) ANALYSIS OF OVERLAPPING ISSUES

The common causes of layout overlapping are:

1. Use of fixed widths and heights.
2. Excessive use of absolute positioning.

3. Missing *flex-wrap*.
4. Large margin and padding values.
5. Images wider than their containers.
6. Missing media queries.
7. Incorrect *z-index* values.
8. Elements removed from the normal document flow.
9. Not using *box-sizing: border-box*.
10. Long text without wrapping.

For example, the following code can cause overlap:

```
.card {
   position: absolute;
width: 500px;
left: 600px; }
```

The element remains in a fixed position even when the screen becomes smaller.

(b) CSS BOX MODEL

Every HTML element is represented as a rectangular box consisting of four parts:

Margin
  Border
    Padding
      Content

Content

The content is the actual text, image, or other information inside the element.

Padding

Padding is the space between the content and the border.

Border

The border surrounds the padding and content.

Margin

Margin is the space outside the border that separates one element from another.

The following rule prevents padding and borders from increasing an element's specified width:

```
* {
   box-sizing: border-box;
}
```

(c) DEBUGGING USING POSITIONING TECHNIQUES

Relative Positioning

Relative positioning moves an element from its original position while preserving its original space.

```
.element {
position: relative;
top: 10px;
}
```

Absolute Positioning

Absolute positioning removes an element from the normal document flow. It should be used only for small decorative elements.

Flexbox

Flexbox is suitable for arranging items in a row or column.

```
.container {
display: flex;
flex-wrap: wrap;
gap: 20px;
}
```

Grid

Grid is suitable for arranging items into responsive rows and columns.

```
.container {
display: grid;
   grid-template-columns:
      repeat(auto-fit, minmax(220px, 1fr));
gap: 20px;
}
```

ALGORITHM

1. Start the program.
2. Identify elements that overlap.
3. Remove unnecessary absolute positioning.
4. Apply *box-sizing: border-box*.
5. Replace fixed widths with percentages and maximum widths.
6. Use Flexbox or Grid for alignment.
7. Add wrapping and suitable spacing.
8. Add a media query for small screens.
9. Resize the browser and verify the corrected layout.
10. Stop the program.

PROGRAM

Save the file as *css_layout_debugging.html*.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">

   <meta name="viewport"
       content="width=device-width, initial-scale=1.0">

   <title>CSS Layout Debugging</title>

   <style>
```

```
    * {         margin: 0;
padding: 0;         box-
sizing: border-box;
    }

    body {
        font-family: Arial, sans-serif;
background-color: #f2f4f7;
color: #222;         line-height: 1.6;
    }

    header {         padding: 20px;
background-color: #263746;
color: white;         text-align:
center;
    }

    nav {         display: flex;
justify-content: center;         flex-
wrap: wrap;         gap: 10px;
padding: 12px;
background-color: #40566b;
    }

    nav a {         padding:
9px 16px;         color:
white;         text-
decoration: none;
border-radius: 4px;
    }

    nav a:hover {
background-color: #ef8354;
    }
```

```
    .page-container {
width: 90%;          max-
width: 1100px;
margin: 30px auto;
    }

    .layout {          display: grid;
grid-template-columns: 2fr 1fr;
gap: 25px;          align-items: start;
    }
    .main-content,      .sidebar
{          padding: 25px;
background-color: white;
border: 1px solid #ddd;
border-radius: 8px;
    }

    .cards {
display: grid;
        grid-template-columns:
            repeat(auto-fit, minmax(180px, 1fr));
gap: 18px;          margin-top: 20px;
    }

    .card {          min-width: 0;
padding: 20px;
background-color: #e8f0f7;
border-radius: 6px;
overflow-wrap: break-word;
    }

    .card h3 {          margin-
bottom: 10px;          color:
#263746;
    }
```

```
    img {
display: block;
width: 100%;
max-width: 100%;
height: auto;
margin-top: 15px;
border-radius: 6px;
    }

    footer {        margin-top:
30px;        padding: 20px;
background-color: #263746;
color: white;        text-align:
center;
    }

    @media screen and (max-width: 700px) {
      .layout {            grid-
template-columns: 1fr;
      }        nav {
flex-direction: column;
text-align: center;
      }

      .page-container {
width: 94%;
      }

      .main-content,
.sidebar {
padding: 18px;
      }
    }
  </style>
```

```
</head>

<body>
<header>
  <h1>Responsive Debugged Layout</h1>
</header>

<nav>
  <a href="#">Home</a>
  <a href="#">Courses</a>
  <a href="#">Services</a>
  <a href="#">Contact</a>
</nav>

<div class="page-container">

  <div class="layout">

    <main class="main-content">
      <h2>Main Content</h2>

      <p>
        This layout uses CSS Grid instead of fixed
positioning. Therefore, the content remains aligned
when the browser is resized.
      </p>

      <div class="cards">
        <div class="card">
          <h3>HTML</h3>
          <p>
            HTML provides the structure of a webpage.
          </p>
        </div>
```

```
            <div class="card">
                <h3>CSS</h3>
                <p>
                    CSS controls webpage layout and appearance.
                </p>
            </div>

            <div class="card">
                <h3>JavaScript</h3>
                <p>
                    JavaScript provides interactive behaviour.
                </p>
            </div>
        </div>

        <img
            src="https://images.unsplash.com/photo-1498050108023-c5249f4df085?auto=format&fit=crop&w=1000&q=80"
            alt="Web development workspace">
    </main>

    <aside class="sidebar">
        <h2>Sidebar</h2>

        <p>
            The sidebar appears next to the main content on
larger screens.
        </p>

        <p>
            On mobile devices, it moves below the main
content without overlapping.
        </p>
    </aside>
```

```
    </div>
</div>
<footer>
    <p>CSS Layout Debugging Experiment</p>
</footer>

</body>
</html>
```

(d) MODIFICATIONS MADE TO PREVENT OVERLAPPING

The following corrections were applied:

1. Fixed widths were replaced with percentage widths.
2. Absolute positioning was removed.
3. CSS Grid was used for the main content and sidebar.
4. *auto-fit* and *minmax()* were used for responsive cards.
5. Responsive images were created using *width: 100%.*
6. *overflow-wrap: break-word* was added for long text.
7. A media query was added for screens below 700 pixels.
8. The sidebar moves below the main content on mobile devices.
9. *gap* was used for consistent spacing.
10. *box-sizing: border-box* was applied to all elements.

TEST CASES

Test Case	Expected Result
Desktop view	Main content and sidebar appear side by side
Tablet view	Layout adjusts without overlapping
Mobile view	Sidebar moves below the main content
Image test	Image remains inside its container
Navigation test	Links change to vertical alignment
Text test	Long words wrap inside the cards
Resize test	No horizontal scrolling or overlap occurs

(e) EVALUATION AND BEST PRACTICES

The corrected layout remains properly aligned on different screen sizes. CSS Grid handles the main structure, while responsive widths and media queries prevent overlapping.

The following best practices should be followed:

1. Avoid fixed widths for major containers.
2. Avoid unnecessary absolute positioning.
3. Use Flexbox and Grid for layout design.
4. Apply *box-sizing: border-box*.
5. Use responsive images.
6. Add media queries for smaller devices.
7. Use *gap* for consistent spacing.
8. Test the webpage at different screen sizes.
9. Use semantic HTML tags.
10. Inspect elements using browser developer tools.

RESULT

Thus, the overlapping webpage layout was successfully debugged using the CSS box model, Grid, Flexbox, responsive dimensions, and media queries. The corrected layout adapts properly during browser resizing.

OVERALL CONCLUSION

The integrated experiments demonstrated responsive webpage design, JavaScript form validation, and CSS layout debugging.

CSS Flexbox, Grid, media queries, and responsive dimensions improve webpage alignment on mobile, tablet, and desktop devices. JavaScript regular expressions prevent invalid information from being submitted. Correct use of the CSS box model and positioning techniques prevents overlapping and provides a consistent user experience.